Supervised Fine-Tuning (SFT)

Teaching LLMs to Formulate Text & Follow Instructions

Yakov (Dima)

September 2026

Supervised Fine-Tuning (SFT)
Teaching LLMs to Formulate Text & Follow Instructions
Yakov (Dima)
September 2026

Two Purposes of SFT

Alignment SFT

Base completion model → general instruction-following assistant.

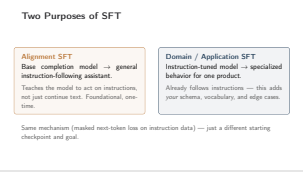
Teaches the model to act on instructions, not just continue text. Foundational, one-time.

Domain / Application SFT

Instruction-tuned model → specialized behavior for one product.

Already follows instructions — this adds *your* schema, vocabulary, and edge cases.

Same mechanism (masked next-token loss on instruction data) — just a different starting checkpoint and goal.



Alignment SFT: Pre-training vs. SFT (The Mindset Shift)

Base LLM — Text Completer

Prompt: Write a python function for quicksort.

Completion: Write a python function for mergesort.
Write a python function for binary search.
Write a python function for...

Chaos pattern: keeps completing the list instead of answering it.

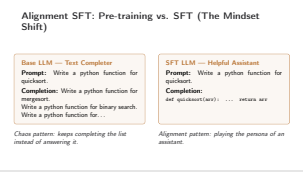
SFT LLM — Helpful Assistant

Prompt: Write a python function for quicksort.

Completion:

```
def quicksort(arr):  ...  return arr
```

Alignment pattern: playing the persona of an assistant.



Domain SFT: An Autonomous-Driving Copilot

General Instruct Model (Alignment SFT only)

Prompt: Pedestrian at crosswalk distracted; truck in bike lane 20m ahead.

Completion: Moderate risk — distracted pedestrian near the crosswalk, truck blocking the bike lane ahead. Recommend slowing down.

Limitation: correct but free-form prose — not parseable by a planner.

Domain-SFT'd Driving Copilot

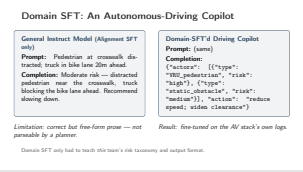
Prompt: (same)

Completion:

```
{"actors": [{"type":
"VRU_pedestrian", "risk":
"high"}, {"type":
"static_obstacle", "risk":
"medium"}], "action": "reduce
speed; widen clearance"}
```

Result: fine-tuned on the AV stack's own logs.

Domain SFT only had to teach *this* team's risk taxonomy and output format.



Training: Same Loss, Adapted Rules

<|user|> What is 2+2? <|assistant|> The answer is 4.

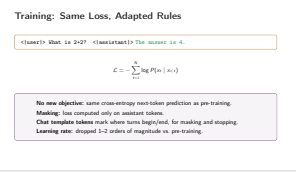
$$\mathcal{L} = -\sum_{t=i}^{N} \log P(x_t \mid x_{<t})$$

No new objective: same cross-entropy next-token prediction as pre-training.

Masking: loss computed only on assistant tokens.

Chat template tokens mark where turns begin/end, for masking and stopping.

Learning rate: dropped 1–2 orders of magnitude vs. pre-training.



The formula: Let $x_{<t}$ denote the model's input context at step t (system, template, user, and prior assistant tokens), and $\bar{x}_t$ be the ground-truth next token from the annotation. If $\mathcal{N}(x_{<t})$ represents the model's predicted probability distribution over the vocabulary, the probability assigned to the correct token is $P(x_t = \bar{x}_t \mid x_{<t}) = \mathcal{N}(x_{<t})[\bar{x}_t]$. In Supervised Fine-Tuning (SFT), the negative log-likelihood loss is computed exclusively over the assistant tokens (starting at index i up to sequence length T):

$$\mathcal{L}_{\text{SFT}} = -\sum_{t=i}^{T} \log \mathcal{N}(x_{<t})[\bar{x}_t]$$

No new objective: same cross-entropy prediction as pre-training, different data.

Masking: loss only on assistant tokens — user/system turns are context, not targets.

Chat template tokens mark turn boundaries, for masking and stopping.

Learning rate: down 1–2 orders of magnitude vs. pre-training (∼2e-5 full FT, ∼1e-4 LoRA).

2026-09-28

Supervised Fine-Tuning (SFT)

The LLM Alignment Pipeline



Pre-training: web-scale raw knowledge and language structure acquisition.

SFT: teaching interaction formats, tasks, and system behaviors.

RLHF/DPO: micro-tuning behavior to meet safety standards and human feedback.

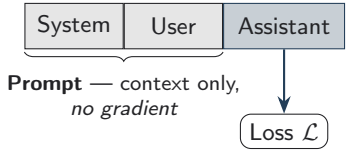
This 3-stage pipeline was introduced by Ouyang et al., *Training language models to follow instructions with human feedback* (InstructGPT, 2022).

The LLM Alignment Pipeline
Pre-training (Base Model) → SFT (Instruction Model) → RLHF / DPO (Human Preferences)
Pre-training: web-scale raw knowledge and language structure acquisition.
SFT: teaching interaction formats, tasks, and system behaviors.
RLHF/DPO: micro-tuning behavior to meet safety standards and human feedback.
This 3-stage pipeline was introduced by Ouyang et al., Training language models to follow instructions with human feedback (InstructGPT, 2022).

The Anatomy of an Instruction Example

ChatML / OpenAI JSON template structure

```
[
  {"role": "system", "content": "You are a helpful coding assistant."},
  {"role": "user", "content": "How do I reverse a list in Python?"},
  {"role": "assistant", "content": "You can reverse a list using
     `list.reverse()` or slicing `[::-1]`."}
]
```



The Gold Target *— loss is calculated ONLY on the assistant response.*

System: defines boundaries/tone.

User: input instructions — part of the context, not the target.

Assistant: the training target — this is what the loss actually optimizes.

The Anatomy of an Instruction Example
ChatML / OpenAI JSON template structure
[
{"role": "system", "content": "You are a helpful coding assistant."},
{"role": "user", "content": "How do I reverse a list in Python?"},
{"role": "assistant", "content": "You can reverse a list using
`list.reverse()` or slicing `[::-1]`."}
]
System
User
Assistant
Prompt — context only, no gradient
Loss $\mathcal{L}$
The Gold Target — loss is calculated ONLY on the assistant response.
System: defines boundaries/tone.
User: input instructions — part of the context, not the target.
Assistant: the training target — this is what the loss actually optimizes.

Purpose of this slide: show the audience what a single SFT training example physically looks like, and set up the key mechanical fact (loss masking) that the next slide's formula depends on.

The code block: this is the raw JSON that instruction datasets are stored in (ChatML / OpenAI's chat format) — not a diagram, an actual example of the data format. Point out it's a list of {role, content} turns, concatenated with special template tokens (not shown here, kept simple) into one flat sequence before it ever reaches the model — that flattening is what the next slide's $x_1, \ldots, x_N$ refers to.

> *system* turn: sets the assistant's persona/behavior for the whole conversation — written once, not something the model is being taught to produce.
>
> *user* turn: the instruction/question — this is input the model must condition on, but never generates itself.
>
> *assistant* turn: the one thing in the whole example the model is actually being trained to produce.

The role diagram (gray System+User blocks braced as "Prompt," colored Assistant block with an arrow down to Loss): the compressed version of the mechanical rule — system+user together form "the prompt," which is fed in as context; only the assistant span contributes to the loss. The arrow deliberately starts *only* at the Assistant block — there is no arrow out of System/User — so the picture itself says "no gradient from the prompt," not just the caption underneath it. Say this out loud as "everything up to the assistant turn is context the model reads; only the assistant turn is what it's graded on."

"The Gold Target" caption: names the assistant turn as the *annotation* — i.e. this is the ground-truth label a human (or a stronger model) wrote, analogous to a label in any supervised-learning setup. Worth explicitly drawing the parallel to classification labels here if the audience needs the supervised-learning framing reinforced.

Bottom bullets: restate the three roles in one line each, mapping directly onto the code block above — use these as the actual talking points per role rather than re-reading the JSON. The key contrast to land is System+User = context/never a training target vs. Assistant = the only thing being optimized; this is exactly what "loss masking" means and is the fact the training-mechanics slide right after this one builds on.

Instruction Tuning Works: The FLAN Result

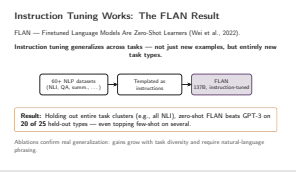
FLAN — Finetuned Language Models Are Zero-Shot Learners (Wei et al., 2022).

Instruction tuning generalizes across tasks — not just new examples, but entirely new task types.



Result: Holding out entire task clusters (e.g., all NLI), zero-shot FLAN beats GPT-3 on **20 of 25** held-out types — even topping few-shot on several.

Ablations confirm real generalization: gains grow with task diversity and require natural-language phrasing.


Instruction Tuning Works: The FLAN Result
FLAN — Finetuned Language Models Are Zero-Shot Learners (Wei et al., 2022).
Instruction tuning generalizes across tasks — not just new examples, but entirely new task types.
60+ NLP datasets (NLI, QA, summ., . . .)
Templated as instructions
FLAN 137B, instruction-tuned
Result: Holding out entire task clusters (e.g., all NLI), zero-shot FLAN beats GPT-3 on 20 of 25 held-out types — even topping few-shot on several.
Ablations confirm real generalization: gains grow with task diversity and require natural-language phrasing.

What this paper is about. FLAN = "Finetuned Language Net." GPT-3 (2020) showed few-shot prompting works but zero-shot is weak. FLAN's fix: instruction-tune a pretrained LM once on many tasks phrased as natural-language instructions, so it learns to *follow instructions* as a skill, not just continue text.

Method (§2.1). 62 labeled NLP datasets (NLI, QA, translation, . . .) → 12 task clusters → ∼10 hand-written NL templates each. Finetune a 137B LM (LaMDA-PT) on the mixture → FLAN.

Eval (§2.2–3). Strict zero-shot: hold out a whole task *cluster* (e.g. no NLI at all), evaluate on it. Beats zero-shot GPT-3 on 20/25 held-out datasets; beats even few-shot GPT-3 on ANLI, RTE, BoolQ, ARC, OpenbookQA, StoryCloze.

Why not just multi-task training (§4.1–4.3). (a) more clusters keeps helping, not saturated; (b) only works at scale — *hurts* models ≤8B (capacity goes to memorizing the ∼40 tasks, not generalizing); (c) dropping the instructions (raw I/O, or a dataset-name tag) performs much worse — the phrasing itself is what's doing the work.

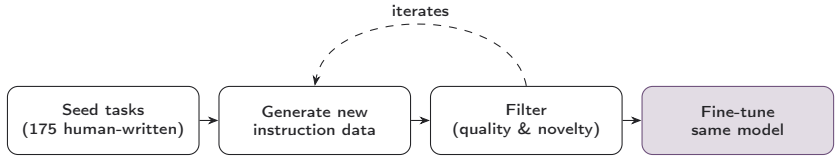
GPT-3 caveat. This GPT-3 (Brown et al., 2020) is pretrained-only, evaluated by prompting only (no gradient updates) — not instruction-tuned. InstructGPT (Ouyang et al., 2022, on the Pipeline slide) applies SFT+RLHF to GPT-3 a year later. FLAN beats the *prompting-only* paradigm of its time, not a rival instruction-tuned model.

Actual contribution: not the benchmark numbers, and not FLAN's data-sourcing trick (repurposed datasets — not how modern pipelines source SFT data, see next slides). It's proof instruction tuning itself induces real generalization to unseen human–model interactions.

Bootstrapping Instructions from the Model Itself: Self-Instruct

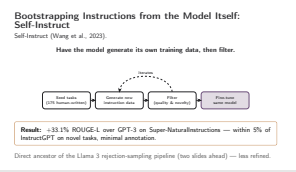
Self-Instruct (Wang et al., 2023).

Have the model generate its own training data, then filter.



Result: +33.1% ROUGE-L over GPT-3 on Super-NaturalInstructions — within 5% of InstructGPT on novel tasks, minimal annotation.

Direct ancestor of the Llama 3 rejection-sampling pipeline (two slides ahead) — less refined.



Purpose: closes the "where does instruction data come from" arc opened by FLAN — FLAN reused existing labeled datasets; Self-Instruct bootstraps from the model's own generations instead. Wang et al. 2023: 175 seed tasks, iteratively expanded, +33.1% ROUGE-L over vanilla GPT-3 on Super-NaturalInstructions, within 5% of InstructGPT_{001}.

ROUGE-L: Recall-Oriented Understudy for Gisting Evaluation, Longest Common Subsequence variant — measures text overlap between generated and reference outputs by finding the longest matching word sequence (order-sensitive, unlike unigram ROUGE). Ranges 0–100; higher is better. SuperNI (Super-NaturalInstructions) uses ROUGE-L because it tests diverse generation tasks, not just classification. Vanilla GPT-3's 6.8 means its zero-shot outputs barely overlap with the gold references; Self-Instruct's 39.9 shows real instruction-following ability.

The filter, precisely (§2): three cheap automatic checks, none of which verify correctness. *Diversity:* new instruction kept only if ROUGE-L vs. every existing instruction is < 0.7 (catches near-dupes, not wrong answers). *Keyword filter:* drop instructions needing images/graphs a text-only LM can't do. *Instance heuristics:* drop exact-dup instances, same-input-conflicting-output pairs, and too-long/short or input-repeating outputs. No reward model, no human judgment, no correctness check anywhere.

Is the data actually correct? No (§3.3). 200 sampled instructions, human-graded: 92% valid instruction, only 58% correct output, only **54% fully valid end to end** — shipped anyway ("the value of the generated data despite the noise"). One-time diagnostic, not a real filter. Bet: CE-loss SFT tolerates a noisy minority, partial-correctness still teaches format, and quality is raised *later* via iteration/distillation (re-scoring outputs with InstructGPT_{003} lifted results 10%) rather than guaranteed upfront.

Where the Data Actually Comes From

Both reports use mostly model-generated data — humans verify or select, not write from scratch.

DeepSeek-V3 — Model Generates, Humans Verify

Reasoning (math/code/logic): generated by **DeepSeek-R1**, an internal AI reasoning model, then rejection-sampled.

Non-reasoning (writing, role-play, QA): generated by **DeepSeek-V2.5**; humans *verify* correctness, don't write it.

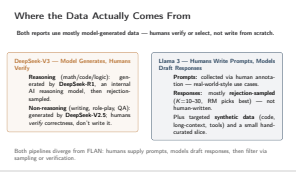
Llama 3 — Humans Write Prompts, Models Draft Responses

Prompts: collected via human annotation — real-world-style use cases.

Responses: mostly **rejection-sampled** (K=10–30, RM picks best) — not human-written.

Plus targeted **synthetic data** (code, long-context, tools) and a small hand-curated slice.

Both pipelines diverge from FLAN: humans supply prompts, models draft responses, then filter via sampling or verification.



Purpose: the concrete, production-scale answer to "where does the data come from," using the same two reports the deck treats as verified sources elsewhere (DeepSeek-V3/Llama 3 case studies two slides ahead cover *quality*, not sourcing).
Sources. DeepSeek-V3 Tech. Report §5.1: reasoning data (math/code/logic) generated by an internal DeepSeek-R1 expert model (own SFT+RL pipeline), rejection-sampled into the final mix; non-reasoning data generated by DeepSeek-V2.5, human-verified for correctness. Llama 3 Herd of Models §4.2.1–4.2.2 (Table 6/7): prompts collected via human annotation; SFT responses mostly rejection-sampled (K=10–30, RM picks best), plus targeted synthetic data where hand-annotation is impractical (e.g. 38K-token long-context examples) and a small hand-curated slice.

Delivery: contrast with FLAN — FLAN assumed labeled datasets already existed; neither lab has that luxury for an open-ended assistant, so both converge instead on: humans supply/vet *prompts*, a model drafts *responses*, rejection sampling or human review filters. Direct descendant of Self-Instruct's generate-then-filter loop (previous slide), just industrialized.

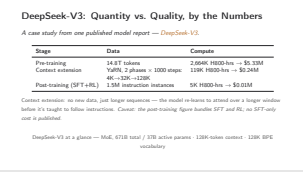
DeepSeek-V3: Quantity vs. Quality, by the Numbers

A case study from one published model report — DeepSeek-V3.

Stage	Data	Compute
Pre-training	14.8T tokens	2,664K H800-hrs → $5.33M
Context extension	YaRN, 2 phases × 1000 steps: 4K→32K→128K	119K H800-hrs → $0.24M
Post-training (SFT+RL)	1.5M instruction instances	5K H800-hrs → $0.01M

Context extension: no new data, just longer sequences — the model re-learns to attend over a longer window before it's taught to follow instructions. *Caveat: the post-training figure bundles SFT and RL; no SFT-only cost is published.*

DeepSeek-V3 at a glance — MoE, 671B total / 37B active params · 128K-token context · 128K BPE vocabulary


DeepSeek-V3: Quantity vs. Quality, by the Numbers
A case study from one published model report — DeepSeek-V3.
Stage | Data | Compute
Pre-training | 14.8T tokens | 2,664K H800-hrs → $5.33M
Context extension | YaRN, 2 phases × 1000 steps: 4K→32K→128K | 119K H800-hrs → $0.24M
Post-training (SFT+RL) | 1.5M instruction instances | 5K H800-hrs → $0.01M
Context extension: no new data, just longer sequences — the model re-learns to attend over a longer window before it's taught to follow instructions. Caveat: the post-training figure bundles SFT and RL; no SFT-only cost is published.
DeepSeek-V3 at a glance — MoE, 671B total / 37B active params · 128K-token context · 128K BPE vocabulary

Purpose: land one number-backed contrast before the mechanics — pre-training and SFT run the *same* loss on the *same* model, but on data from opposite regimes. This slide stays inside a single report so every ratio is an apples-to-apples comparison; the next slide switches to Llama 3 to show *how* curation is actually done.

Sources. DeepSeek-V3 Technical Report, Table 1 (§1) — 2664K / 119K / 5K H800 GPU-hours for pre-training / context extension / post-training, priced at $2 per H800 GPU-hour, $5.576M total. Pre-training data: §4 (14.8T "high-quality and diverse tokens"). Context extension: §4.3 — YaRN, two 1000-step phases, 4K→32K then 32K→128K. SFT data: §5 — "we curate our instruction-tuning datasets to include 1.5M instances spanning multiple domains." Model specs: §1/§4.1 — MoE, 671B total / 37B active params, 128K context, 128K BPE vocab.

Delivery: read the ratio, not the digits — 500× and seven orders of magnitude are the point; the exact GPU-hours just show it's a real reported figure, not a hand-wave.

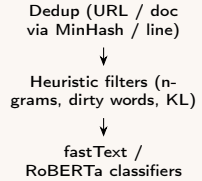
Caveat: nobody publishes an SFT-only budget — production pipelines interleave SFT and RL, so "post-training is cheap" is honest at the stage level, not per-algorithm. Next slide: if it's that cheap, what's actually hard about it?

What "Curated" Actually Means

A case study from one published model report — Llama 3.

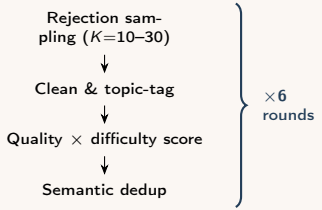
Pre-training Data

Scraped from the web — bulk, automatic, corpus-level

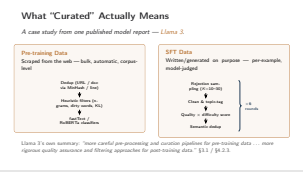


SFT Data

Written/generated on purpose — per-example, model-judged



Llama 3's own summary: *"more careful pre-processing and curation pipelines for pre-training data . . . more rigorous quality assurance and filtering approaches for post-training data."* §3.1 / §4.2.3.



Purpose: both columns are Llama 3 alone — shows *how* its "more rigorous" claim is implemented, step by step.
Left (§3.1): *Dedup* — URL/line matching + MinHash. *Heuristic filters* — n-gram/dirty-word counts flag spam; token-distribution KL flags non-natural-language text. *fastText/RoBERTa classifiers* — learned scorers (RoBERTa trained on Llama 2's own quality judgments) keep only higher-quality pages.
Right (§4.2.3): *Rejection sampling (K=10–30)* — per prompt, sample K outputs from the current best chat checkpoint, RM scores all K, keep the top pick; later rounds add system prompts to steer tone before scoring. *Clean & topic-tag* — rules + a fine-tuned Llama 3 8B classifier. *Quality×difficulty* — RM top-quartile score × Instag difficulty, so hard examples survive. *Semantic dedup* — RoBERTa clustering removes near-copies. Whole chain repeats **6 rounds**, each round's model generating the next round's candidates.
Secretly RL? Yes: best-of-K-then-imitate is the same family as best-of-N policy improvement / STaR / RAFT / ReST — RM picks the best current-policy sample, SFT imitates it. *Not* PPO/DPO-style: no gradient through reward/sampling, no value function, and top-1-of-K discards all reward signal below the winner — "RL-flavored data generation feeding a supervised objective."
Guarantees correct data? No. Humans mainly verify prompts/preference pairs, not every SFT sample — a statistical filter at scale, not a per-example guarantee (the paper admits its RM- and Llama-based scores "have high disagreement rates"). Same bet as Self-Instruct, stricter and iterated 6×: tolerate noise, raise quality via iteration.

Delivery: name the funnel shape (bulk/automatic left vs. model-judged right); hold RL-framing/no-guarantee point in reserve for questions.

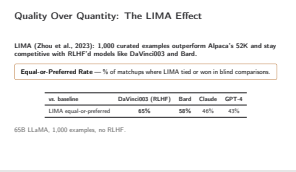
Quality Over Quantity: The LIMA Effect

LIMA (Zhou et al., 2023): 1,000 curated examples outperform Alpaca's 52K and stay competitive with RLHF'd models like DaVinci003 and Bard.

Equal-or-Preferred Rate — % of matchups where LIMA tied or won in blind comparisons.

vs. baseline	DaVinci003 (RLHF)	Bard	Claude	GPT-4
LIMA equal-or-preferred	**65%**	**58%**	46%	43%

65B LLaMA, 1,000 examples, no RLHF.



Full context (condensed from slide). LIMA is a 65B LLaMA model fine-tuned on just 1,000 meticulously curated prompt/response pairs, with no RLHF at all. It beats a 52×-larger, uncurated set (Alpaca's 52,000 self-instruct examples) and stays competitive with RLHF'd frontier models. The core takeaway: quantity alone doesn't buy alignment quality — careful curation matters more than dataset size.
Equal-or-Preferred Rate (full definition). Annotators see two anonymous responses per prompt and pick the better one, or call it a tie. "X% equal-or-preferred" means LIMA tied or won in X% of matchups.
Sources. Zhou et al., *LIMA: Less Is More for Alignment* (2023), §4.2 / abstract — equal-or-preferred rates: 65% vs. DaVinci003, 58% vs. Bard, 46% vs. Claude, 43% vs. GPT-4. Alpaca comparison (52,000 self-instruct examples, not raw web data) is qualitative in the paper — no exact % given, so it's stated as text above rather than in the table.

Delivery: the punchline up top is the actual takeaway — LIMA doesn't uniformly "win," it's *competitive* despite 1–2 orders of magnitude less data and no RLHF. Don't oversell the table as LIMA beating everyone; it loses more often than not to Claude/GPT-4, just not by a landslide.

After SFT: Why RL Still Helps

Chu et al., 2025 — SFT and RL operate on different signal frequencies.

SFT: Low-Frequency Structure

Learns broad behavioral patterns (instruction-following across tasks).

But overfits to training data's high-frequency details.

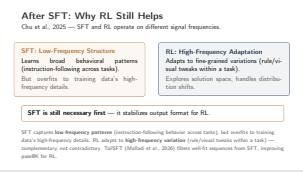
RL: High-Frequency Adaptation

Adapts to fine-grained variations (rule/visual tweaks within a task).

Explores solution space, handles distribution shifts.

SFT is still necessary first — it stabilizes output format for RL.

SFT captures **low-frequency patterns** (instruction-following behavior across tasks), but overfits to training data's high-frequency details. RL adapts to **high-frequency variation** (rule/visual tweaks within a task) — complementary, not contradictory. TailSFT (Malladi et al., 2026) filters well-fit sequences from SFT, improving pass@K for RL.

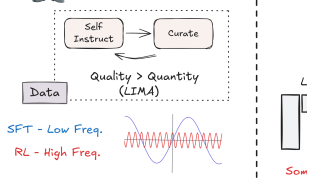


Full context (condensed from slide). This slide explains the next arrow on the pipeline slide: why alignment doesn't stop at SFT. The signal-frequency framing resolves the apparent contradiction with FLAN: SFT generalizes at *low frequency* (captures the broad pattern of instruction-following across wildly different task types, as FLAN showed), but overfits at *high frequency* (memorizes the specific rules/visuals in the training distribution and can't adapt when you tweak them). RL with outcome-based reward operates on the high-frequency band — it explores the solution space and adapts to rule/visual variants within the same task that SFT treats as out-of-distribution noise. Complementary strengths: SFT gives the coarse-grained behavioral structure, RL fine-tunes adaptation to local variation. But SFT is still necessary first: it stabilizes the model's output format (the low-frequency structure), which is what lets subsequent RL training work at all.

TailSFT details (condensed from landscapenote). Recent continuation work: TailSFT (Malladi et al., 2026) filters already-well-fit sequences *out* of SFT itself, preserving more of the output-coverage (pass@K) that RL needs — up to +17% pass@16, and +4% pass@1 after subsequent GRPO.

Purpose: this is a bridge slide, not part of the Alignment-SFT data-curation arc above — it answers "why does the pipeline diagram have an arrow from SFT to RLHF/DPO at all," setting up whatever RLHF/DPO content follows.

Sources. Chu et al., *SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-*



Supervised Fine-Tuning (SFT)

What Alignment SFT + RL Don't Give You

Alignment SFT (low-frequency)

- General instruction-following
- Recognize requests vs. continuations
- Safe, helpful persona

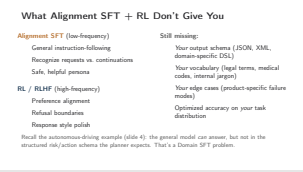
RL / RLHF (high-frequency)

- Preference alignment
- Refusal boundaries
- Response style polish

Still missing:

- *Your* output schema (JSON, XML, domain-specific DSL)
- *Your* vocabulary (legal terms, medical codes, internal jargon)
- *Your* edge cases (product-specific failure modes)
- Optimized accuracy on *your* task distribution

Recall the autonomous-driving example (slide 4): the general model *can* answer, but not in the structured risk/action schema the planner expects. That's a Domain SFT problem.



Point: Alignment SFT teaches "follow instructions"; RL teaches "be helpful and safe." Neither teaches "output in *this* JSON schema" or "use *these* domain terms correctly." That's the gap Domain SFT fills.

The two-purposes framing from slide 2 lands here: we've covered purpose #1 (alignment); now we're entering purpose #2 (domain/application specialization).

Domain Adaptation Still Helps

Key finding *(Gururangan et al., 2020 — "Don't Stop Pretraining"):*
Even after broad pretraining, **continued domain- or task-specific training measurably improves downstream performance.**

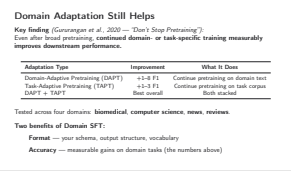
Adaptation Type	Improvement	What It Does
Domain-Adaptive Pretraining (DAPT)	+1–8 F1	Continue pretraining on domain text
Task-Adaptive Pretraining (TAPT)	+1–3 F1	Continue pretraining on task corpus
DAPT + TAPT	Best overall	Both stacked

Tested across four domains: **biomedical**, **computer science**, **news**, **reviews**.

Two benefits of Domain SFT:

Format — your schema, output structure, vocabulary

Accuracy — measurable gains on domain tasks (the numbers above)


Domain Adaptation Still Helps
Key finding (Gururangan et al., 2020 — "Don't Stop Pretraining"):
Even after broad pretraining, continued domain- or task-specific training measurably improves downstream performance.
Adaptation Type | Improvement | What It Does
Domain-Adaptive Pretraining (DAPT) | +1–8 F1 | Continue pretraining on domain text
Task-Adaptive Pretraining (TAPT) | +1–3 F1 | Continue pretraining on task corpus
DAPT + TAPT | Best overall | Both stacked
Tested across four domains: biomedical, computer science, news, reviews.
Two benefits of Domain SFT:
Format — your schema, output structure, vocabulary
Accuracy — measurable gains on domain tasks (the numbers above)

Citation: Gururangan et al., "Don't Stop Pretraining: Adapt Language Models to Domains and Tasks," ACL 2020.

The paper shows that domain-adaptive pretraining (DAPT) on unlabeled domain text, and task-adaptive pretraining (TAPT) on the task's own unlabeled corpus, both help—and they stack. This is the empirical case for why Domain SFT isn't just about format, but about actual task performance.

Alternative 1: Prompt Engineering

Idea: Instead of retraining, steer the model via instructions and examples in the prompt.

Aspect	Prompt Engineering	Domain SFT
Upfront Cost	$0 — immediate via API	High — data + training
Per-Request Cost	High — examples every call	Minimal — short prompts
Format Reliability	Can drift, model may ignore	Baked into weights
Update Speed	Instant	Requires retraining

When to use:

- Low volume, high variance tasks
- Rapid iteration / experimentation phase
- No training infrastructure available

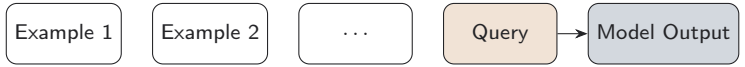
Prompt engineering is mostly an alternative to *domain* SFT (an already-instruction-following model).
It can approximate alignment too, but fragile and paid per-request.

Alternative 1: Prompt Engineering
Idea: Instead of retraining, steer the model via instructions and examples in the prompt.
Aspect | Prompt Engineering | Domain SFT
Upfront Cost | $0 — immediate via API | High — data + training
Per-Request Cost | High — examples every call | Minimal — short prompts
Format Reliability | Can drift, model may ignore | Baked into weights
Update Speed | Instant | Requires retraining
When to use:
Low volume, high variance tasks
Rapid iteration / experimentation phase
No training infrastructure available
Prompt engineering is mostly an alternative to domain SFT (an already-instruction-following model).
It can approximate alignment too, but fragile and paid per-request.

Key tradeoff: Zero upfront cost, but you pay on every request (tokens for examples), and the model can ignore or misinterpret your instructions. SFT flips this: high upfront cost, but behavior is durable and prompts stay short.

Alternative 2: In-Context Learning (ICL)

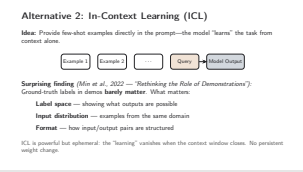
Idea: Provide few-shot examples directly in the prompt—the model "learns" the task from context alone.



Surprising finding *(Min et al., 2022 — "Rethinking the Role of Demonstrations"):* Ground-truth labels in demos **barely matter**. What matters:

- **Label space** — showing what outputs are possible
- **Input distribution** — examples from the same domain
- **Format** — how input/output pairs are structured

ICL is powerful but ephemeral: the "learning" vanishes when the context window closes. No persistent weight change.


Alternative 2: In-Context Learning (ICL)

Idea: Provide few-shot examples directly in the prompt—the model "learns" the task from context alone.

Example 1 Example 2 · · · Query Model Output

Surprising finding (Min et al., 2022 — "Rethinking the Role of Demonstrations"): Ground-truth labels in demos barely matter. What matters:

- Label space — showing what outputs are possible
- Input distribution — examples from the same domain
- Format — how input/output pairs are structured

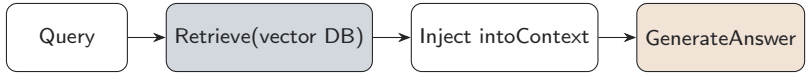
ICL is powerful but ephemeral: the "learning" vanishes when the context window closes. No persistent weight change.

Citation: Min et al., "Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?" EMNLP 2022.

The paper shows that randomly replacing labels barely hurts performance—the model primarily uses the demos to understand format and distribution, not to learn input-label mappings. This explains why ICL works but also why it's limited.

Alternative 3: Retrieval-Augmented Generation (RAG)

Idea: Automate what goes into the context by retrieving relevant documents at inference time.



Key properties *(Lewis et al., 2020):*

Non-parametric memory: knowledge lives in the index, not the weights

Instant updates: swap documents, knowledge changes immediately

Provenance: can cite which documents informed the answer

Reduced hallucination: model grounds responses in retrieved text

RAG excels at **knowledge**-intensive tasks (facts, QA). For **behavior** changes (output format, reasoning style), SFT is usually needed.

Alternative 3: Retrieval-Augmented Generation (RAG)
Idea: Automate what goes into the context by retrieving relevant documents at inference time.
Query
Retrieve(vector DB)
Inject intoContext
GenerateAnswer
Key properties (Lewis et al., 2020):
Non-parametric memory: knowledge lives in the index, not the weights
Instant updates: swap documents, knowledge changes immediately
Provenance: can cite which documents informed the answer
Reduced hallucination: model grounds responses in retrieved text
RAG excels at knowledge-intensive tasks (facts, QA). For behavior changes (output format, reasoning style), SFT is usually needed.

Citation: Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020.

RAG combines parametric (model weights) and non-parametric (document index) memory. The key insight: knowledge can be updated without retraining by just updating the index.

RAG Variants: Active, Hierarchical, Graph-Based

The basic RAG pattern has many extensions:

Variant	Key Idea	Best For
Self-RAG (Asai et al., 2023)	Model decides *when* to retrieve; critiques own outputs	Reducing unnecessary retrieval
FLARE (Jiang et al., 2023)	Retrieves *during* generation when confidence drops	Long-form generation
RAPTOR (Sarthi et al., 2024)	Hierarchical tree of summaries, not flat chunks	Multi-hop reasoning
Graph RAG (Edge et al., 2025)	Entity graph + community summaries	Complex entity relationships

All variants share the core idea: inject external knowledge at inference time rather than baking it into weights. Choice depends on your retrieval patterns and reasoning complexity.

RAG Variants: Active, Hierarchical, Graph-Based
The basic RAG pattern has many extensions:
Variant | Key Idea | Best For
Self-RAG (Asai et al., 2023) | Model decides when to retrieve; critiques own outputs | Reducing unnecessary retrieval
FLARE (Jiang et al., 2023) | Retrieves during generation when confidence drops | Long-form generation
RAPTOR (Sarthi et al., 2024) | Hierarchical tree of summaries, not flat chunks | Multi-hop reasoning
Graph RAG (Edge et al., 2025) | Entity graph + community summaries | Complex entity relationships
All variants share the core idea: inject external knowledge at inference time rather than baking it into weights. Choice depends on your retrieval patterns and reasoning complexity.

These variants address different limitations of basic RAG:

– Self-RAG: not every query needs retrieval
– FLARE: retrieval should happen mid-generation, not just at the start
– RAPTOR: flat chunks miss hierarchical structure
– Graph RAG: entity relationships matter